

Assignment 6: Bayes and Naive Bayes Classifiers

CS201L: Artificial Intelligence Laboratory
Indian Institute of Technology, Dharwad

12 - Feb - 2026

Problem Statement

In this lab, you will address the problem of Pattern Classification, which is the task of identifying to which of a set of categories a new observation belongs. You will utilize the Activity Detection dataset (10,299 samples, 561 features) to classify human movements into six daily categories: LAYING, STANDING, SITTING, WALKING, WALKING UPSTAIRS, and WALKING DOWNSTAIRS. The objective is to compare the performance of Naive Bayes and Bayes classifiers across four variations of the dataset.

Dataset Description

The Human Activity Recognition database was built from recordings of 30 participants performing activities of daily living (ADL) while carrying a waist-mounted smartphone with embedded inertial sensors. Each subject performed six activities:

- WALKING
- WALKING UPSTAIRS
- WALKING DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The smartphone (Samsung Galaxy S II) captured:

- 3-axial linear acceleration (accelerometer)
- 3-axial angular velocity (gyroscope)

at a constant sampling rate of 50Hz. The signals were preprocessed using noise filtering and segmented into fixed-width sliding windows of 2.56 seconds with 50% overlap (128 readings per window). From each window, a vector of features was extracted from both time and frequency domains.

Dataset Properties

- Total subjects: 30
- Activities: 6 (multiclass)
- Total features: 561 (numerical)
- Feature domains: Time-domain and Frequency-domain
- Sensors: Accelerometer, Gyroscope
- Target attribute: Activity
- Data format: CSV file
- Dataset files: original activity-detection.csv has been prepared into individual csv files, refer to the section on ‘Prepared Dataset Details’.

Attribute Information

For each record in the dataset, the following information is provided:

- Triaxial acceleration from the accelerometer (total acceleration)
- Triaxial body acceleration (body motion component)
- Triaxial angular velocity from the gyroscope
- Gravity acceleration components
- 561-feature vector extracted from time and frequency domains
- Activity label (multiclass target)
- Subject identifier (ID of participant)

Dataset Statistics

Property	Value
Total Samples	10,299
Total Features	561
Target Classes	6
Missing Values	0
Data Types	float64 (561), int64 (1), object (1)

Table 1: Overall Dataset Information (Activity Detection)

Activity	Sample Count	Percentage (%)
LAYING	1,944	18.88
STANDING	1,906	18.51
SITTING	1,777	17.25
WALKING	1,722	16.72
WALKING UPSTAIRS	1,544	14.99
WALKING DOWNSTAIRS	1,406	13.65
Total	10,299	100.00

Table 2: Activity-wise Sample Distribution

Prepared Dataset Details

You are provided with 12 CSV files representing 4 dataset variations, each split into Training (60%), Validation (20%), and Testing (20%):

- Original Data: ‘Activity_Train.csv’, ‘Activity_Validation.csv’, ‘Activity_Test.csv’.
- Standardized Data: ‘Activity_Scaled_Train.csv’, ‘Activity_Scaled_Validation.csv’, ‘Activity_Scaled_Test.csv’.
- PCA (All Components): ‘Activity_PCAAll_Train.csv’, ‘Activity_PCAAll_Validation.csv’, ‘Activity_PCAAll_Test.csv’.
- PCA (99% Variance): ‘Activity_PCA99_Train.csv’, ‘Activity_PCA99_Validation.csv’, ‘Activity_PCA99_Test.csv’.

Note: the ‘subject’ attribute has been removed, since it is a class attribute and not a predictive attribute.

Your task is to implement the following classifiers on this dataset:

- **Bayes** uses unimodal Gaussian density function, and assumes that each sample follows a multivariate Gaussian distribution per class.
- **Naive Bayes** has the same properties and assumptions as Bayes but with added assumption that features are statistically independent.

Part A: Naive Bayes Classifier

(Weightage: 30%)

The Naive Bayes classifier assumes that data follow a Gaussian distribution and all features are statistically independent.

A library implementation of Naive Bayes classifier is available in the ‘scikit-learn’ package,

1. Use `fit()` function from `'scikit-learn'` package to train the Naive Bayes classifier.
2. Use `predict()` function from `'scikit-learn'` package to predict labels.

Note: look at the sample code below on how to import and use Naive Bayes from `'scikit-learn'`.

For the four dataset variation,

- Original Data: train the Naive Bayes classifier on `'Activity_Train.csv'`, and predict the class labels for `'Activity_Validation.csv'` and `'Activity_Test.csv'`.
- Standardized Data: train the Naive Bayes classifier on `'Activity_Scaled_Train.csv'`, and predict the class labels for `'Activity_Scaled_Validation.csv'` and `'Activity_Scaled_Test.csv'`.
- PCA (All Components): train the Naive Bayes classifier on `'Activity_PCAAll_Train.csv'`, and predict the class labels for `'Activity_PCAAll_Validation.csv'`, `'Activity_PCAAll_Test.csv'`.
- PCA (99% Variance): train the Naive Bayes classifier on `'Activity_PCA99_Train.csv'`, and predict the class labels for `'Activity_PCA99_Validation.csv'` and `'Activity_PCA99_Test.csv'`.

Finally, evaluate and report the confusion matrix, accuracy, precision, recall, and F1-score (macro and micro) for all four test/validation sets.

Implementation Tasks

- Load the training, validation, and test data for each of the four variations.
- Use `'GaussianNB()'` to initialize the model.
- Perform model training using the `'fit()'` function on the training feature vectors and target vectors.
- Predict class labels for the validation and test data using the `'predict()'` function.
- Evaluate and report the confusion matrix, accuracy, precision, recall, and F1-score (macro and micro) for all four test sets.

Sample Naive Bayes Code:

```
1 from sklearn.naive_bayes import GaussianNB
2
3 # Step 1: Initialize the Gaussian Naive Bayes model
4 naiveBayes = GaussianNB()
5
```

```

6  # Step 2: Model Training (Parameter estimation)
7  # Parameter estimation or model training is performed using the
   ↪ fit function
8  naiveBayes.fit(X_train, y_train)
9
10 # Step 3: Predict class labels for test or validation data
11 # Use the predict function to generate class levels
12 y_test_pred = naiveBayes.predict(X_test)

```

Part B: Implementation of Bayes Classifier

(Weightage: 40%)

The Bayes classifier uses unimodal Gaussian density function, and assumes that data follows a multivariate Gaussian distribution per class and applies Bayes' theorem for classification.

Algorithm

Training

1. Load training, validation, and test datasets. Separate the input feature vectors ($\mathbf{x}$) from the target class labels ($\mathbf{y}$).
2. Class Separation: Group the training feature vectors into their respective classes C_i (where $i = 1, 2, \dots, M$ & M is the no. of classes).
3. Parameter Estimation: For each class C_i , compute the following from the training data:
 - 3.1 Mean Vector ($\hat{\boldsymbol{\mu}}_i$): The average of all feature vectors in class i .
 - 3.2 Covariance Matrix ($\hat{\boldsymbol{\Sigma}}_i$): Representing the relationships between features within class i .
 - 3.3 Prior Probability ($P(C_i)$): Calculated as:

$$P(C_i) = \frac{\text{Number of samples in class } i}{\text{Total training samples}}$$

Testing and Validation

1. For each sample $\mathbf{x}$ in the test data,
 - 1.1 Likelihood Computation: find the likelihood $p(\mathbf{x}|\hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i)$ using the multivariate normal distribution density function (`multivariate_normal.pdf` from '`scipy.stats`'). Ensure that you set '`allow_singular=True`' to handle potentially singular covariance matrices.

1.2 Posterior Probability Calculation: Compute the posterior for each class:

$$P(\hat{\mu}_i, \hat{\Sigma}_i | \mathbf{x}) = \frac{p(\mathbf{x} | \hat{\mu}_i, \hat{\Sigma}_i) P(C_i)}{\sum_{i=1}^M p(\mathbf{x} | \hat{\mu}_i, \hat{\Sigma}_i) P(C_i)}$$

where $P(\hat{\mu}_i, \hat{\Sigma}_i | \mathbf{x})$ is the posterior probability, $p(\mathbf{x} | \hat{\mu}_i, \hat{\Sigma}_i)$ is the likelihood, $P(C_i)$ is the prior, and $\sum_{i=1}^M p(\mathbf{x} | \hat{\mu}_i, \hat{\Sigma}_i) P(C_i)$ is the total sum probability.

1.3 Decision Rule: Assign the class label C_i that yields the highest posterior probability,

$$\text{Class label} = \underset{i}{\operatorname{argmax}} P(\hat{\mu}_i, \hat{\Sigma}_i | \mathbf{x})$$

1.4 Append '*Class label*' to the class prediction vector $\hat{C}$

2. Return the predicted class vector $\hat{C}$.

You can also refer to the class slides for specific parts of the algorithm.

Part C: Bayes Classifier Evaluation

(Weightage: 30%)

Apply your implementation of Bayes classifier for the four dataset variation,

- Original Data: train the Bayes classifier on '`Activity_Train.csv`', and predict the class labels for '`Activity_Validation.csv`' and '`Activity_Test.csv`'.
- Standardized Data: train the Bayes classifier on '`Activity_Scaled_Train.csv`', and predict the class labels for '`Activity_Scaled_Validation.csv`' and '`Activity_Scaled_Test.csv`'.
- PCA (All Components): train the Bayes classifier on '`Activity_PCAAll_Train.csv`', and predict the class labels for '`Activity_PCAAll_Validation.csv`', '`Activity_PCAAll_Test.csv`'.
- PCA (99% Variance): train the Bayes classifier on '`Activity_PCA99_Train.csv`', and predict the class labels for '`Activity_PCA99_Validation.csv`' and '`Activity_PCA99_Test.csv`'.

Finally, evaluate and report the confusion matrix, accuracy, precision, recall, and F1-score (macro and micro) for all four test/validation sets.

Deliverables

Provide a Jupyter Notebook containing:

1. Code for all parts.
2. Plots & Summaries: confusion matrices, and any other visualizations you deem useful.
3. A summary table of metrics for each method.

Zip the notebook and data into a file named `your.rollnumber_Assignment6.zip` and submit it to Moodle.

References

- UCI Machine Learning Repository – Human Activity Recognition Using Smartphones Dataset.
- Scikit-Learn documentation: <https://scikit-learn.org/stable/>.
- SciPy statistics module: <https://docs.scipy.org/doc/scipy/reference/stats.html>.